DAY 13 TECHNICAL ACTIVITY REPORT

Python Database Connectivity & Dynamic CRUD Automation

Programmatic MySQL Drivers, Cursor Iteration, and Student Management System Architecture

TRAINEE	MENTOR	INSTITUTION	DATE & STATUS
Benatic Rithan B	Rathees G	KIT, Coimbatore Dept of CSE	22 June 2026 ● Completed Successfully

1. EXECUTIVE SUMMARY

This technical report documents the implementation of programmatic backend data-layer integration completed during Day 13 of the Database and Backend track. The primary target focused on building full database connectivity by linking Python execution environments with a local MySQL server via low-level network socket drivers. Rather than performing manual isolated commands through external CLI software, an enterprise-patterned `Student Management System` was engineered. The tool abstracts standard SQL commands directly within unified object-oriented methods while securely managing network connections, statements, transactions, and transaction state rolls.

2. API SOCKET CONNECTIVITY & STATE COMPLEXITY ANALYSIS

To ensure stable network data routing without risking connection memory leaks or structural transaction lockouts, specific driver interface routines were applied:

- **Driver Handshaking Protocols:** Establishes an TCP/IP communication pipe connecting Python runtime layers with MySQL host ports, managing credentials safely via isolated connection instances.
- **Cursor State Allocation:** Instantiates logical execution contexts on the database server, allowing the app to process resulting sets sequentially while optimizing local client-side memory.
- **Parameterized Query Execution:** Sanitizes parameters to separate structural statements from dynamic user values, neutralizing SQL injection vulnerabilities.
- **Deterministic ACIDs & Transaction Saves:** Groups structural data alterations under strict manual commit rules, using fallback rollback steps to protect consistency if network faults interrupt queries.

3. STRUCTURAL MAPPING MATRIX

SYSTEM PIPELINE	PYTHON CONNECTOR MECHANISM	DATABASE SERVER SIDE ACTION	DATA VULNERABILITY SAFEGUARD
Connection Init	`mysql.connector.connect()`	Allocates network worker thread resources for incoming sockets.	Prevents unauthorized access errors.
Query Sanitization	`cursor.execute(query, params)`	Compiles structural query logic separate from variable inputs.	Blocks SQL Injection attacks entirely.
Record Extraction	`cursor.fetchall()` / `fetchone()`	Streams matching index records out to client application lists.	Avoids client-side memory thrashing.
Transaction Seal	`connection.commit()` / `rollback()`	Flushes log buffers to confirm permanent row operations on disk.	Prevents incomplete database states.

4. TECHNICAL DATABASE IMPLEMENTATION

The following object-oriented Python implementation encapsulates full relational connectivity, programmatic cursors, parameter sanitization, and structured transaction error boundaries:

```python
import mysql.connector
from mysql.connector import Error

class StudentManagementSystem:
    def __init__(self, host, user, password, db):
        try:
            self.conn = mysql.connector.connect(host=host, user=user, password=password, database=db)
            self.cursor = self.conn.cursor(dictionary=True)
        except Error as e: print(f"Connection breakdown: {e}")

    def create_student(self, name: str, email: str, major: str) -> bool:
        query = "INSERT INTO students (name, email, major) VALUES (%s, %s, %s)"
        try:
            self.cursor.execute(query, (name, email, major))
            self.conn.commit()
            return True
        except Error: self.conn.rollback(); return False

    def get_students_by_major(self, major: str) -> list:
        query = "SELECT * FROM students WHERE major = %s"
        self.cursor.execute(query, (major,))
        return self.cursor.fetchall()

    def update_student_email(self, student_id: int, new_email: str) -> bool:
        query = "UPDATE students SET email = %s WHERE student_id = %s"
        try:
            self.cursor.execute(query, (new_email, student_id))
            self.conn.commit()
            return True
        except Error: self.conn.rollback(); return False

    def close_pipeline(self):
        if self.conn.is_connected():
            self.cursor.close(); self.conn.close()
```

5. WORKSPACE ENVIRONMENT STATE MOCKUPS

PROGRAMMATIC DATA EXTRACTION EXECUTION LOGS (CONSOLE OUTPUT VECTOR)

```
>>> sys = StudentManagementSystem(host="localhost", user="root", password="***", db="CampusDB")
>>> sys.create_student("Benatic Rithan", "benatic@kit.edu.in", "Computer Science Engineering")
[SUCCESS] Transaction committed. Row inserted matching Auto-ID: 1

>>> sys.get_students_by_major("Computer Science Engineering")
[{'student_id': 1, 'name': 'Benatic Rithan', 'email': 'benatic@kit.edu.in', 'major': 'Computer Science
Engineering'}]
```

VERSION CONTROL TRACK METRICS (GITHUB SUBMISSION REPOSITORY STATE)

```
        Origin: remote upstream master branch
        [Commit Ref: #DB-D13-P61] feat: configure mysql-connector client network pipelines with auto-
retry
        [Commit Ref: #DB-D13-P62] feat: implement param-sanitized CRUD operations to block injection
exploits
        [Commit Ref: #DB-D13-P63] feat: add transaction safety boundaries utilizing atomic commit-
rollback logic
```

6. LEARNING OUTCOMES

- **Programmatic Driver Orchestration:** Mastered client-server network socket setups connecting Python code blocks directly with active relational database instances.
- **Cursor Lifecycle Regulation:** Leveraged automated cursors to execute database queries and fetch large datasets efficiently without memory bottlenecks.
- **Injection Mitigation Mastery:** Implemented parameterized variable bindings across script components to completely protect queries from dangerous string manipulation attacks.
- **Atomic State Preservation:** Engineered conditional transactional blocks using try-except frameworks to guarantee data safety across incomplete batch records.